

Step 3 — Class Design

The Cinema Ticket Booking System is divided into **Data Classes**, **Service Classes**, and a **UI Class**.

Design Rule: Data classes store information and maintain their own state. Service classes perform business operations. The UI class handles user interaction.

DATA CLASSES

```
|
|— Movie
|— Seat
|— Screen
|— Cinema
|— Show
|— ShowSeat
|— Customer
|— Booking
```

SERVICE CLASSES

```
|
|— Payment
|   |— UpiPayment
|   |— CardPayment
|   |— CashPayment
|
|— PriceCalculator
|— TicketPrinter
|— BookingService
```

UI CLASS

```
|
|— CinemaMainMenu
```

3.1 Data Classes

3.1.1 Movie

Purpose

Stores basic information about a movie.

Data Members

```
string title;  
string language;  
int duration;
```

Methods

```
string getTitle();  
string getLanguage();  
int getDuration();
```

Responsibility

`Movie` only stores movie information. It does not handle booking, payment, pricing, or seat management.

3.1.2 Seat

Purpose

Represents one physical seat inside a screen.

Data Members

```
string seatNumber;  
SeatType seatType;
```

SeatType

```
SILVER  
GOLD  
PLATINUM
```

Methods

```
string getSeatNumber();  
SeatType getSeatType();
```

Important

Seat does not store booking status.

The same physical seat can be:

```
10:00 AM → Available  
2:00 PM → Booked  
7:00 PM → Available
```

Therefore, availability is handled by ShowSeat .

3.1.3 Screen

Purpose

Represents one cinema screen and its physical seats.

Data Members

```
int screenNumber;  
vector<Seat> seats;
```

Methods

```
int getScreenNumber();  
const vector<Seat>& getSeats();
```

Relationship

```
Screen  
  ↓  
Contains  
  ↓  
Seats
```

3.1.4 Cinema

Purpose

Represents the cinema and its screens.

Data Members

```
string name;  
vector<Screen> screens;
```

Methods

```
string getName();  
const vector<Screen>& getScreens();
```

Relationship

```
Cinema  
  ↓  
Contains  
  ↓  
Screens
```

3.1.5 Show

Purpose

Represents one scheduled screening of a movie.

Example:

```
Movie   : Avengers  
Screen  : 2  
Time    : 7:00 PM
```

Data Members

```
Movie* movie;  
Screen* screen;  
string showTime;  
vector<ShowSeat> showSeats;
```

Methods

```
Movie* getMovie();  
Screen* getScreen();  
string getShowTime();  
const vector<ShowSeat>& getShowSeats();
```

Responsibility

Show connects:

```
Movie + Screen + Show Time
```

It does not calculate prices or process payments.

3.1.6 ShowSeat

Purpose

Represents the availability of a particular physical seat for one specific show.

Data Members

```
Seat* seat;  
ShowSeatStatus status;
```

ShowSeatStatus

```
AVAILABLE  
BOOKED
```

Methods

```
bool book();  
bool cancel();  
bool isAvailable();  
Seat* getSeat();  
ShowSeatStatus getStatus();
```

Why ShowSeat is required

Suppose the cinema has physical seat A1 .

```
A1
|
|— 10:00 AM → Available
|— 2:00 PM  → Booked
|— 7:00 PM  → Available
```

Therefore:

```
Seat
↓
Physical chair

ShowSeat
↓
Availability of that chair for one show
```

This prevents incorrect seat-booking information.

3.1.7 Customer

Purpose

Stores customer information.

Data Members

```
string name;
string phone;
```

Methods

```
string getName();
string getPhone();
```

The customer does not directly perform booking or payment operations.

3.1.8 Booking

Purpose

Stores the details and current state of a customer's booking.

Data Members

```
string bookingId;  
Customer* customer;  
Show* show;  
vector<ShowSeat*> seats;  
double totalAmount;  
BookingStatus status;
```

BookingStatus

```
PENDING  
CONFIRMED  
CANCELLED
```

Methods

```
bool confirm();  
bool cancel();  
  
string getBookingId();  
Customer* getCustomer();  
Show* getShow();  
const vector<ShowSeat*>& getSeats();  
double getTotalAmount();  
BookingStatus getStatus();
```

Booking Lifecycle

```
PENDING  
  ↓  
Payment Successful  
  ↓  
CONFIRMED  
  ↓  
Cancellation  
  ↓  
CANCELLED
```

A booking becomes `CONFIRMED` only after successful payment.

3.2 Service Classes

Service classes perform business operations using the domain classes.

3.2.1 Payment

Purpose

Defines a common interface for different payment methods.

```
class Payment {  
public:  
    virtual bool pay(double amount) = 0;  
    virtual ~Payment() = default;  
};
```

The payment hierarchy is:

```
Payment  
|  
├── UpiPayment  
├── CardPayment  
└── CashPayment
```

UpiPayment

```
class UpiPayment : public Payment {  
public:  
    bool pay(double amount) override;  
};
```

CardPayment

```
class CardPayment : public Payment {  
public:  
    bool pay(double amount) override;  
};
```

CashPayment

```
class CashPayment : public Payment {  
public:  
    bool pay(double amount) override;  
};
```

For this academic project, these classes can simply simulate successful or failed payments. Real card credentials should not be stored directly.

3.2.2 PriceCalculator

Purpose

Calculates the total ticket price based on the selected seat types.

Data Members

```
double silverPrice;  
double goldPrice;  
double platinumPrice;
```

Method

```
double calculate(const vector<ShowSeat*>& seats);
```

Example

```
A1 → SILVER    → ₹150  
A2 → GOLD      → ₹250  
B1 → PLATINUM  → ₹350  
  
Total = ₹750
```

`PriceCalculator` only calculates the price. It does not book seats or process payments.

3.2.3 TicketPrinter

Purpose

Displays the details of a confirmed booking.

Method

```
void printTicket(const Booking& booking);
```

Example:

```
-----  
                CINEMA TICKET  
-----  
  
Booking ID : B101  
Customer   : Rahul  
Movie      : Avengers  
Show Time  : 7:00 PM  
Seats      : A1 A2  
Amount     : ₹400  
Status     : CONFIRMED  
  
-----
```

`TicketPrinter` only handles ticket presentation.

3.2.4 BookingService

Purpose

Coordinates the complete booking and cancellation process.

`BookingService` is the main coordinator of the system. It does not perform every operation itself. It delegates work to specialized classes.

Data Members

```
vector<Show*> shows;  
vector<Booking*> bookings;  
PriceCalculator priceCalculator;
```

Methods

```
Booking* startBooking();  
bool cancelBooking(string bookingId);  
Show* findShow(string showId);  
string generateBookingId();
```

Responsibilities

```
Select Show  
  ↓  
Select Seats  
  ↓  
Check Availability  
  ↓  
Calculate Price  
  ↓  
Create PENDING Booking  
  ↓  
Process Payment  
  ↓  
Confirm Booking  
  ↓  
Print Ticket
```

3.3 UI Class

CinemaMainMenu

Purpose

Provides the user interface and accepts user input.

Data Member

```
BookingService* bookingService;
```

Methods

```
void showMenu();  
int takeUserInput();
```

Example Menu

```
=====
CINEMA TICKET SYSTEM
=====
```

1. View Shows
2. Book Ticket
3. Cancel Ticket
4. Exit

Enter choice:

CinemaMainMenu does not contain:

```
Price calculation
Payment logic
Seat-booking logic
Booking state management
```

It sends the user's request to `BookingService` .

3.4 Complete Class List

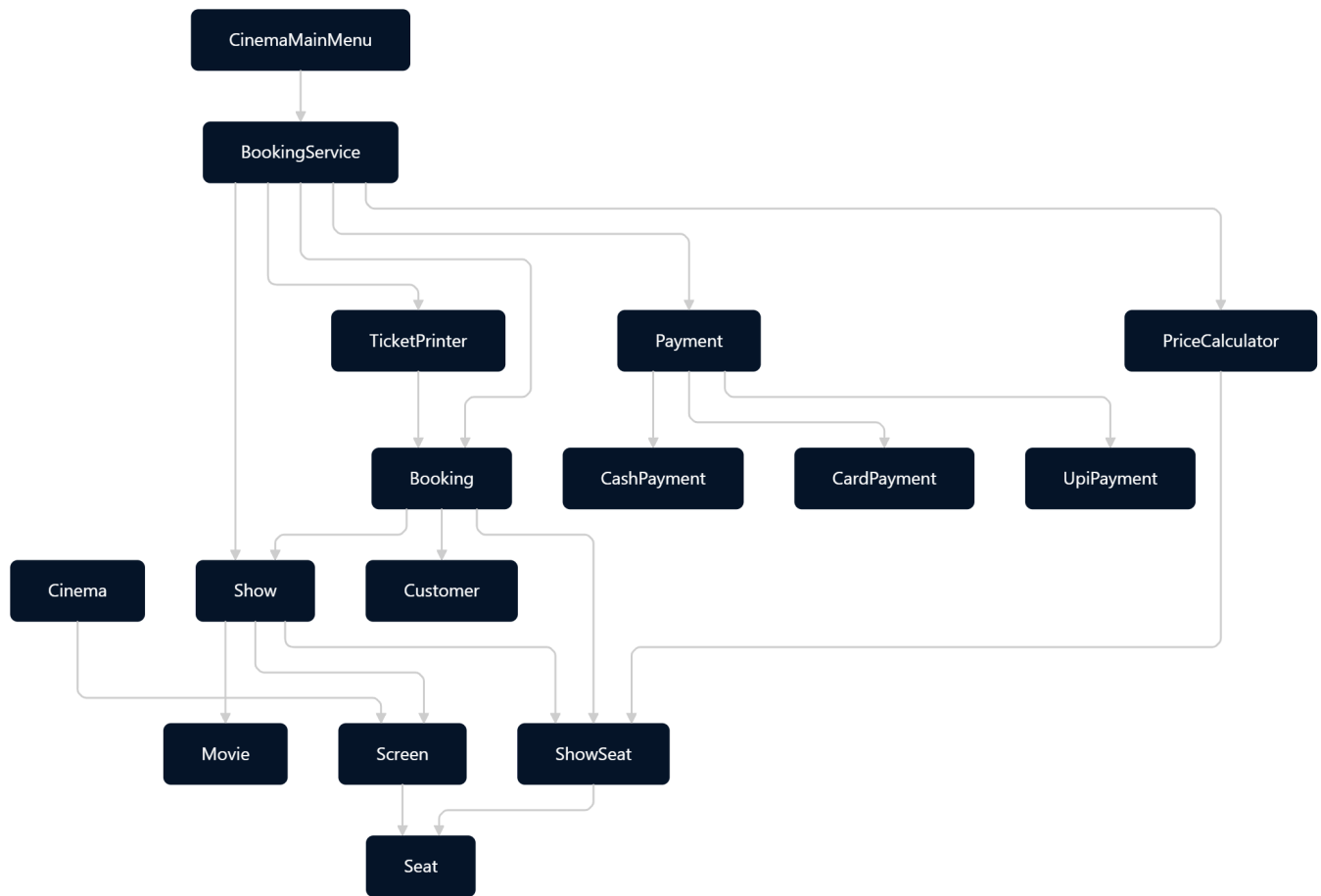
Class	Responsibility
Movie	Stores movie information
Seat	Represents a physical seat
Screen	Contains physical seats
Cinema	Contains screens
Show	Represents a scheduled movie screening
ShowSeat	Manages seat availability for a particular show
Customer	Stores customer information
Booking	Stores booking details and status
Payment	Defines the common payment interface
UpiPayment	Handles UPI payment
CardPayment	Handles card payment
CashPayment	Handles cash payment
PriceCalculator	Calculates ticket price
TicketPrinter	Displays/prints the ticket

Class	Responsibility
BookingService	Coordinates booking and cancellation
CinemaMainMenu	Handles user interaction

3.5 Class Responsibility Table

Class	Main Responsibility
Movie	Movie information
Seat	Physical seat
Screen	Collection of seats
Cinema	Collection of screens
Show	Movie + Screen + Show Time
ShowSeat	Seat availability for a particular show
Customer	Customer information
Booking	Booking information and status
Payment	Common payment abstraction
UpiPayment	UPI payment
CardPayment	Card payment
CashPayment	Cash payment
PriceCalculator	Ticket price calculation
TicketPrinter	Ticket presentation
BookingService	Booking workflow coordination
CinemaMainMenu	User interface

3.6 Final Class Design



Golden Rule

Movie	→ Movie information
Seat	→ Physical seat
Screen	→ Contains seats
Cinema	→ Contains screens
Show	→ Movie + Screen + Show Time
ShowSeat	→ Seat availability for a show
Customer	→ Customer information
Booking	→ Booking information and status
Payment	→ Payment abstraction
PriceCalculator	→ Ticket price calculation
TicketPrinter	→ Ticket presentation
BookingService	→ Booking coordination
CinemaMainMenu	→ User interaction

Each class has one clear responsibility, and business operations are handled by service classes rather than data classes.